



How to be a Better Software Engineer in the World of AI Agents

Cheaper code makes engineering judgment more valuable 

 GitHub /  Sponsor /  Mastodon /  X /  Bluesky /  PyPI /  LinkedIn



This talk is about staying a strong engineer while coding agents do more of the typing. Agents made code cheaper to produce, and checking that the code is correct costs what it did, so more of our work goes into that checking.

Over the next forty-five minutes I will show you what got cheaper, what the evidence says in both directions, the four skills you keep, the practices that help, and how to bring a team along without turning it into a mandate. Agents are good at some things and bad at others, and I will show you both.

The QR code goes to the slides and all the references in them, so scan it now if you want to follow along.

Bernát Gábor



Source: Bernát Gábor. License: CC BY 4.0.

-  [gaborbernat](https://github.com/gaborbernat) · bernat.tech
-  PyPA and tox-dev: [virtualenv](#), [tox](#), [pipx](#), [platformdirs](#), [pipdeptree](#)
-  Bloomberg, Developer Experience
-  GitHub Copilot since early 2025; Claude Code since late 2025; Codex since Feb 2026

I maintain virtualenv and pipx under the Python Packaging Authority, and tox, platformdirs, and pipdeptree under tox-dev. Users download them about 800 million times a month. My day job is developer experience at Bloomberg.

I started using GitHub Copilot on my open source work in early 2025, moved to Claude Code late that year, and added Codex in February 2026. Since then about 90% of the open source code I have shipped came out of an agent. This talk is about the work I did around the agent to make that code fit to merge.

The cost moved

Boundaries that cost too much to cross
got cheap.

Source: T. Allom and J. Tingle, "Power loom weaving", in E. Baines, History of the Cotton
Manufacture, 1835, via Wikimedia Commons. License: public domain.



Bloomberg
© 2014 Bloomberg Finance L.P.
All rights reserved.

The cost of producing software changed, and I want to show you the change before I argue anything from it. I will use CPython, PyPI, one of my own projects, Shopify, and Bun, and then an economist from 1865 to explain why the pattern repeats.

🔗 AI is Already Inside CPython

discuss.python.org, Jun-Jul 2026

"Likely >90% of my own contributions in the past year have flowed through Claude Code."¹

Gregory P. Smith, core developer

"would take months or even years to do all this work manually"

Serhiy Storchaka, on tkinter and curses

¹ [discuss.python.org: should Claude Code's usage be described?](https://discuss.python.org/should-claude-code-s-usage-be-described?), Jun-Jul 2026

I will start inside our own ecosystem.

At the end of June, Skip Montanaro pulled CPython's main branch after some time away and saw a long list of recent commits with Claude as co-author. He asked on discuss.python.org whether the project should document that somewhere. In early July Gregory Smith answered that more than 90% of his own contributions in the past year went through Claude Code.

Serhiy Storchaka wrote that AI helped him close gaps in tkinter and curses that would have taken him months or years by hand, and that he reviews each change before he commits it.

Skip asked how to record agent use. A few posters were sceptical, and the core developers who answered were relaxed about it. Two of the people who maintain the reference implementation of the language already work this way.



The PSF Names Agents in PyPI's Growth

"A lot more Python being written, a lot more agents installing packages on someone's behalf, and a lot more CI runs..."

Jacob Coffee, PSF Director of Engineering, Aug 2026¹

6B+

requests/day

~10 PB

egress/day

+69%

AWS spend, Jul 2026 vs. Jul 2025

3.1x

new projects/month, Dec 2024 to Apr 2026²

~1.5 full-time staff on PyPI development and security, +1 on support

AWS spend +40% over 12 months · files and malware reports 2.5x, Dec 2024 to Apr 2026

¹ Jacob Coffee: How AWS powers PyPI and the PSF, Aug 2026

² Mike Fiedler, "Limiting vectors and incentives for abuse", Packaging Summit, May 2026

One thread is an anecdote, so I will show the scale underneath it.

Jacob Coffee, the PSF's Director of Engineering, wrote in August about PyPI's infrastructure. He names agents among the causes of the growth: people write more Python, agents install more packages on someone's behalf, and CI runs more often.

PyPI serves over six billion requests a day, and Fastly delivers almost all of the downloads. The AWS part of the bill keeps growing. Mike Fiedler showed the Packaging Summit that new projects a month grew 3.1 times since December 2024, and malware reports grew with them.

▶ AWS credits pay almost all of that bill, so the PSF pays in staff time. Jacob is the only person working full time on infrastructure. Feature work, a rising volume of security reports and day-to-day maintenance fall to about one and a half full-time staff, with one more person on support requests. I will come back to that constraint when we get to review.

pipdeptree in Rust: Same Tree, 4.6x Faster

 **Contract:** same dependency tree as the Python engine

 **Evidence:** 90 Python + 372 Rust tests · 100% of 4,827 Rust lines¹ ·

`unsafe_code = "forbid"`²

 **Result:** 4.6x end-to-end, 260 packages, mean of 20 runs³ · 5 to 15x at 5,000 packages¹

I defined correctness before the agent wrote any code.

2 to 3.3x at 100 packages · peak memory 17 to 25% lower (PR #618)

```
Same dependency tree. Full 260-package environment:
latest pure Python  481.7 ms
latest Rust-backed  103.8 ms

4.6x faster · mean of 20 runs · same uv environment
$
```

Source: Bernát Gábor, recorded run of pipdeptree 3.1.1 (last pure-Python release) versus 4.2.5 (latest Rust-backed) on the same 260-package uv environment, CPython 3.14.7. License: CC BY 4.0.

¹ pipdeptree 4.0.0, Jul 2026, PR #618; timings include Python startup

² pipdeptree Cargo.toml: unsafe_code = "forbid"

³ My recording on this slide: pipdeptree 3.1.1 versus 4.2.5, hyperfine, mean of 20 runs, Sep 2026

Now I will take one project from that ecosystem, mine, so you can hold me to it, and I will come back to it in each part of this talk.

pipdeptree prints the dependency tree of a Python environment, and its engine used to be pure Python. I replaced that engine with a Rust extension behind PyO3 and kept a small Python layer for the CLI. I read Rust, though I would not call myself an expert. The agent wrote most of the code. I asked the agent to explain any part I did not follow, and then I checked that explanation against what the tests showed.

I supplied the boundary. The Rust engine has to produce the same dependency tree as the old Python engine, which makes the old behaviour the specification. I re-ran both on a larger environment for this talk. The trees matched, but the new engine prints some version specifiers in normalised form, such as <2.0a0 where the old one printed <2.0a.0. The tests had no package with an unnormalised specifier, so that detail changed in a part of the contract the tests did not spell out. Coverage stays at 100% on both sides. The crate sets unsafe_code to forbid, so the compiler rejects any unsafe block we try to add; the unsafe code at the boundary with CPython belongs to PyO3. CI does not run Miri or a fuzzer, and I show why that gap matters with the Bun story in the next part. With those constraints in place, I could review the work.

End to end, the Rust version runs 4.6 times faster in the recording, averaged over repeated runs on my laptop, so treat the exact ratio as approximate. On large synthetic environments the benchmarks show five to fifteen times, and peak memory drops by 17 to 25%. Those timings include Python startup.

► The rewrite worked because I defined correctness before the agent wrote any code. Emma Smith covered Rust in CPython itself at four o'clock.

↔ Cost Shifts Reverse Settled Decisions

Shopify, 2020: React Native · Jan 2025: reaffirmed · Sep 2026: Swift and Kotlin

"Coding agents changed what it costs to build mobile apps twice."¹

Shop app: proof of concept to app stores in **12 weeks**

Gated by tests · visual review · 2 adversarial reviewers · human sign-off

¹ Shopify: native is now the future of mobile at Shopify, Sep 2026

Shopify is the same story at company scale.

Shopify bet on React Native in 2020, and it worked for them, with one codebase and no chase for parity between platforms. In January 2025 they reaffirmed it in public. In September they announced a move back to Swift and Kotlin.

► They give the reason in the first line of the post: coding agents changed what it costs to build mobile apps twice. An agent can implement a feature on Android using the iOS version as the reference, and the other way round, so they no longer had to choose the architecture by the cost of parity. Native still means two codebases, and agents made the second one cheaper to keep in step.

The Shop app went from proof of concept to a rebuilt native app in the stores in twelve weeks. Read the section they titled "preventing slop". Pointing an agent at the React Native code and asking for a one-shot port did not work, and neither did freezing the design into specs and task files up front. The team got results from a loop of small checkpoints, each one gated by tests, a visual comparison with the running app, two adversarial code reviewers, and a human who signed off.

Your own choice of mobile stack is a separate question. My point is that a core assumption changed and the team re-opened a settled decision. Noticing that took human judgment, and the port followed from it.

People Steer and Verify; Agents Own Volume

Bun, Zig to Rust, 11 days: ¹ 1 engineer · 64 agents · +1M lines

Anthropic owns Bun; pre-release Claude model

~1.39M assertions · 0 tests skipped or deleted

Crawshaw: 25% → 90% of his code in a year; reads 95, writes 5 ²

Terry Tao: two dozen maths applets revived ³

~50 workflows · 4 worktrees · 6,778 commits · peak ~1,300 lines a minute · 2+ adversarial reviewers per change

¹ Jarred Sumner: Rewriting Bun in Rust, Jul 2026

² Crawshaw: eight more months of agents, Feb 2026

³ Terry Tao: old and new apps via coding agents, Jul 2026

Bun is a larger case. Jarred Sumner and a fleet of agents rewrote it from Zig to Rust in eleven days. The post discloses that Anthropic bought Bun last December, that Jarred works there, and that he used a pre-release Claude model. A diff of that size is too large for one person to read, and Jarred's post explains what made it safe to merge.

► The Bun team wrote its test suite in TypeScript, so it does not depend on the language Jarred was replacing. Jarred ran the same suite against the Zig original and the Rust port, and he skipped or deleted zero tests. Adversarial reviewers checked each change before commit. The approach has a cost, which I show in the next part.

I have two smaller examples in the same direction. Within a year, David Crawshaw went from the model writing 25% of his code to 90%, and he now reads far more code than he writes. Terry Tao revived old Java applets for mathematics and built new ones. He chose what to build and judged whether the mathematics came out right.

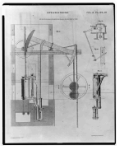
At each scale the human owns direction and verification, and the agent owns volume.



Cheaper Coal Power Raised Coal Use: The Jevons Paradox

*"It is wholly a confusion of ideas to suppose that the economical use of fuel is equivalent to a diminished consumption. The very contrary is the truth."*¹

William Stanley Jevons, *The Coal Question*, 1865



Less coal per engine; more coal burned

Nadella, Jan 2025: Jevons for AI usage² · Ages 22 to 25 in AI-exposed jobs: 19% below less-exposed peers³

¹ Jevons, *The Coal Question*, ch. VII
² GeekWire on Nadella, Jan 2025
³ Stanford Digital Economy Lab: *Canaries in the Coal Mine*, Aug 2026 update
Images: Jevons portrait, 1877; Watt engine plate, J. Robison, 1822; Wikimedia Commons. License: public domain.

Bloomberg
© 2025 Bloomberg Finance LP
All rights reserved.

I do not know what this means for developer jobs in the long run, and I found no study that settles it. I find some optimism in one historical pattern, with a caveat.

In 1865 William Stanley Jevons described a pattern in coal use. ► Watt's engine did the same work on a fraction of the coal that older engines burned, so you would expect coal use to fall. Coal use climbed, because cheaper power made iron works, factories, and machine shops worth running, and more of them burned coal.

Software may work the same way. Plenty of software, such as internal automation, goes unbuilt because it costs too much. If such tools take a week to build and not a month, teams build more of them.

► Satya Nadella reached for Jevons in January 2025, after DeepSeek, about how much AI people would use. That is a claim about AI usage, and more code does not guarantee more engineers. In its August update the Stanford Digital Economy Lab reports employment for 22 to 25 year olds in AI-exposed jobs 19% below where it would be had it kept pace with less-exposed peers, because firms hire fewer of them, and the authors say their data is descriptive and does not show the cause. I take Jevons as a reason for optimism about demand and the hiring data as a reason to watch how juniors get in.

The bar did not

Producing code got cheaper, and proving it correct costs as much as before.

Up to now I covered the falling cost of producing code. The bar for what counts as correct stayed where it was, so in this part I look at what the measurements say, on both sides, and at how open source projects responded.

Adoption is Routine, Trust is Partial



79%¹
use AI weekly



65%¹
of AI code accepted

Stack Overflow 2025: 84% use or plan to use AI tools.³



96%²
do not *fully* trust it is correct



48%²
always check before commit

¹ Faros Q3 2026, telemetry from 22,000 developers (vendor)

² Sonar State of Code, fielded Oct 2025, published Jan 2026, n=1,149 (vendor)

³ Stack Overflow survey, 2025

The top row comes from Faros, which sells engineering analytics and measures developers through telemetry. 79% of them use at least one AI tool weekly, and they accept most of what it suggests. The Stack Overflow survey puts the share higher. By either number, most developers use AI tools as part of ordinary work.

► The bottom row is a survey from Sonar, which sells code verification. 96% say they do not fully trust that AI code is functionally correct, and 48% say they always check it before they commit. “Not fully” includes people who trust the code most of the time, and people outside the “always” group may still check most commits. The survey does not say how the two groups overlap, so I will not claim that it does.



Output Rose in Field Studies; Speed Stays Unsettled

+24% merged PRs · Microsoft, 4 months ¹

+39% merges, no rise in reverts · 32 firms ²

METR 2026: 18% (returning) / 4% (new) faster, both intervals cross zero ³

Two Copilot studies

55.8% faster · 1 lab task, 95 freelancers, 2023 ⁴

+26% tasks · 4,867 developers, 3 field experiments ⁵

¹ Murphy-Hill et al., Microsoft, Jul 2026, observational
² Sarkar, Chicago Booth, May 2026, author visited Cursor
³ METR, Feb 2026, 57 developers
⁴ Peng et al., Feb 2023
⁵ Cui et al., Feb 2026, three field experiments

I start with the evidence that agents help.

Microsoft gave Claude Code and Copilot CLI to tens of thousands of engineers early this year. Adopters merged about 24% more pull requests than a synthetic control predicted, and the lift held for as long as the authors watched. The authors note that a merged pull request does not equal delivered value, and Microsoft sells Copilot, so keep both in mind.

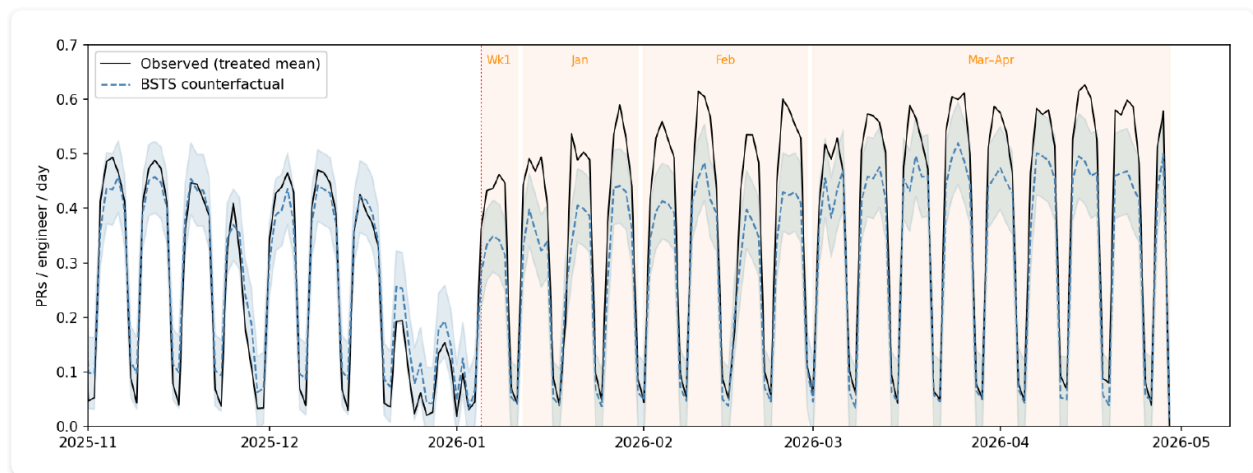
A Chicago Booth study of firms around the release of Cursor's agent found 39% more merges and no rise in reverts. The author was a visiting researcher at Cursor, and he notes that the study does not measure long-term quality.

METR reran the experiment I describe later in this section. The new estimates point toward a speedup, but both confidence intervals cross zero, and METR calls the data "only very weak evidence", because many developers refused to take tasks without AI.

► The "55.8% faster" figure shows up in many talks and posts. It comes from a lab study of freelancers writing one HTTP server. The field experiments at Microsoft, Accenture and a Fortune 100 company are a separate, much larger study, and they found 26% more completed tasks. Both results hold; they measure different things, and articles merge them into one.

Merged Pull Requests at Microsoft

Adopters (solid) against a synthetic control built from non-adopters (dashed), Nov 2025 to Apr 2026.



Source: Murphy-Hill, Butler, Savelieva (Microsoft), "Adoption and Impact of Command-Line AI Coding Agents", Fig. 6, arXiv 2607.01418. License: CC BY 4.0, chart title removed.

The chart shows the Microsoft result. The solid line is engineers who adopted the agents, the dashed line is what the model predicts they would have merged without them, and the gap after January is the 24%. The deep dips are weekends. As the authors note, they counted output and did not measure value.

The Costs, Measured

Velocity faded, complexity stayed: warnings +30%, complexity +41%¹

Graded vs. merged: ~24 points below the grader; maintainers merged 68% of human patches²

Review falls behind: PRs merged without review +76.3%, up from +31.3%³ · copy-paste 8.3% → 12.3%⁴

Felt vs. measured: 19% slower, felt 20% faster⁵

4 maintainers reviewed the METR PRs · GitClear to 2024: moved lines 25% → under 10%

¹ He et al., CMU, MSR 2026
² METR, Mar 2026, 296 agent PRs
³ Faros Q3 2026, vendor telemetry
⁴ GitClear, Feb 2025, 211M changed lines (vendor)
⁵ METR, Jul 2025

For the costs, I apply the same check on what each study counted.

Carnegie Mellon compared open source projects that adopted Cursor with similar projects that did not. Lines added jumped at first and then fell back, while static-analysis warnings and code complexity went up and stayed up; warnings rose about 30% and complexity about 41%. The added complexity then predicted slower work later on. This one is peer reviewed and independent.

METR asked maintainers of scikit-learn, Sphinx and pytest to review agent pull requests that passed the SWE-bench grader, and the maintainers' merge rate came out about 24 points below the grader's pass rate. In a control run, the same maintainers merged 68% of the original human patches. Human review is noisy too, and a patch that passes the tests can still fail review.

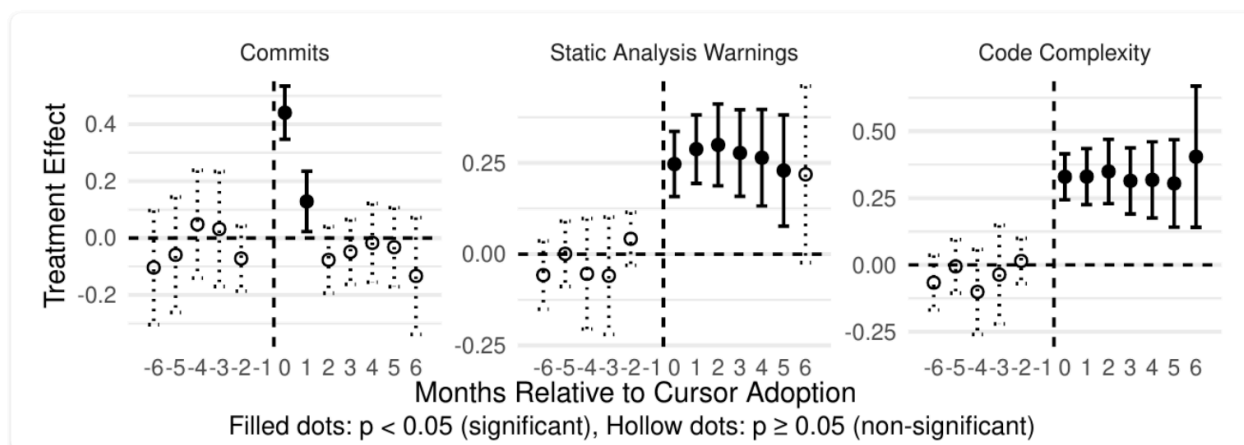
Faros, the vendor from the adoption slide, reports a 76.3% rise in pull requests merged with no review, up from the 31.3% rise in its report six months earlier. That figure is a growth rate; Faros gives no share of all pull requests. Faros also found that pull requests got bigger.

GitClear, also a vendor, found copy-pasted lines rising from 8.3% to 12.3%, and moved lines, their proxy for refactoring, falling from 25% to under 10% by 2024.

► In early 2025 METR timed experienced open source developers using the tools of that time. They took 19% longer with AI and believed they had been 20% faster. The sample is small and the tools are old, and the result still shows a gap between feeling and measurement.

Cursor Adoption: Speed Fades, Complexity Stays

Effect by month relative to adoption. Filled dots are significant.

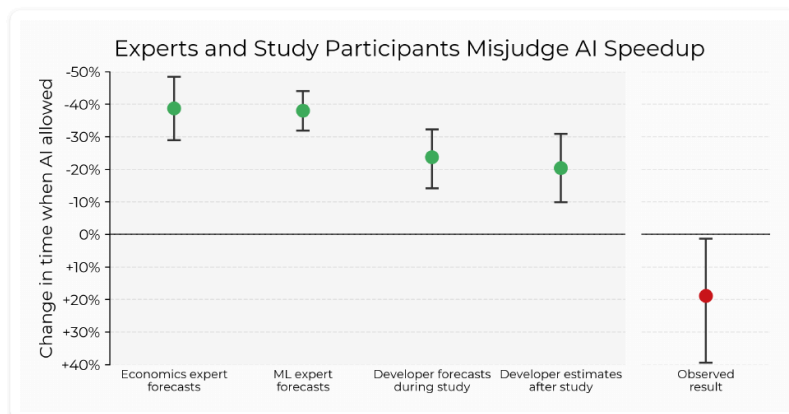


Source: He, Miller, Agarwal, Kästner, Vasilescu, "Speed at the Cost of Quality", MSR 2026, Fig. 3, arXiv 2511.04427, License: CC BY 4.0, cropped to three of five panels.

This chart shows the Carnegie Mellon result. Commits jump in the month a project adopts Cursor and fall back to the baseline by the second month. Static analysis warnings and code complexity rise at the same point; warnings stay up through month five, and complexity through month six. These are open source projects on 2025 tools, so treat it as one data point. The chart shows speed and quality moving in different directions.

⌚ Forecasts Against the Stopwatch

METR, early 2025: 16 experienced developers, Cursor with Claude 3.5 and 3.7 Sonnet. Negative means faster.



Source: Becker, Rush, Barnes et al. (METR), "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity", Fig. 1, arXiv 2507.09089, License: CC BY 4.0.

Economists, machine learning experts, and the developers themselves all expected a speedup of 20 to 40%. On the stopwatch, the tasks took 19% longer. I distrust self-reported numbers because of that gap between belief and measurement. The same group's 2026 rerun pointed toward a speedup, which I showed a few slides ago.



Four Studies, Four Units

Study	Unit counted	Result	Tools
Microsoft, 2026	Merged pull requests	+24%	2026 agents
METR, 2025 to 2026	Time on a fixed task	19% slower to ~18% faster (n.s.)	2025 and 2026
CMU, 2026	Warnings and complexity	+30% and +41%	Cursor, 2025
METR survey ¹	Self-reported speed	3x	2026

Each study counts a different unit. Check the unit before you quote the number.

¹ METR self-report survey, May 2026, n=349, median self-reported speed 3x

You will find studies that say agents make developers faster and studies that say they make them slower, and most of them measured something real. Put the gains and the costs side by side and read the second column of this table, the thing each study counts. Microsoft counted merged pull requests. METR timed a fixed task, first on 2025 tools and then on 2026 tools. Carnegie Mellon counted warnings and complexity over months. A survey asks people how they feel, and METR’s own survey this May found a median self-reported speedup of 3 times, while its 2025 timing study found a 40-point gap between feeling and measurement.

- ▶ Pick a different unit and you get a different answer. Counting pull requests rewards small pull requests and optional work, timing a task holds the scope fixed, and counting warnings shows costs months later. Check the unit before you quote the number to your manager.

My current reading is that output volume goes up, time on a fixed task remains an open question, and quality costs appear later in the few studies that look for them.

The Quality Mirage

94%

rate AI code *higher* quality than human code
at review¹

78%

report more production incidents¹

86%

report more senior-engineer rework¹

Reads well at review; costs more in production.

62% say their teams often ship AI code without line-by-line checks

¹ New Relic State of AI Coding, Jun 2026, vendor-commissioned survey of 200 US tech leaders, manager level and up, self-reported

Bloomberg
©2026 Bloomberg Finance LP
All rights reserved.

These numbers come from a survey, so I name its source first. New Relic commissioned it, New Relic sells observability, and the respondents are 200 US technology leaders, manager level and above, reporting on their own organisations.

94% of them rate AI-generated code higher quality than human code when they review it.

- ▶ 78% of the same managers report more incidents once that code ships, and most also report more rework for their senior engineers.
- ▶ Reviewers who check for plausibility pass code that reads well and still costs more in production. 62% also say their teams often ship AI code without line-by-line checks.



Passing Tests Does Not Prove Soundness

14 May: the rewrite merges. Hours later:

"PathString::slice dangling reference UB - add Miri to CI"

UB reachable from safe Rust¹; full suite passed, nothing skipped²

17 May: Miri test script added, related aliasing bug fixed³

¹ [bun#30719](#), opened 14 May 2026, closed 17 May 2026
² [Jarred Sumner: Rewriting Bun in Rust](#), Jul 2026
³ [bun#30876](#), `bun run rust:miri` and `HiveArray` aliasing fix

The Bun story from the start of the talk has a second half, and it belongs in this section.

The rewrite merged on the fourteenth of May. Hours later an outside user filed an issue: a dangling reference in `PathString`, undefined behaviour reachable from safe Rust. Safe Rust should rule out that class of bug.

► That code passed the full test suite I showed you earlier, with nothing skipped, and it was still unsound. Tests prove behaviour on the cases someone wrote down, and undefined behaviour can hide outside those cases.

Three days later Jarred closed the issue with a change that added a script to run Miri, the Rust interpreter that detects undefined behaviour, and fixed a related aliasing bug it found. That change did not run Miri in CI and left that step to a follow-up. Two weeks after that, the Bun team deleted `PathString`. A bot opened a different fix that marked the API unsafe; the maintainers closed it without merging. Jarred wrote: "please do continue to file new issues about any bugs or unsound behavior you find."

There is a larger argument about that rewrite, and some of it is partisan. I will not settle it from up here; my point is narrower. A person caught this after the tests passed, and the lasting fix was a tool built for that class of bug. `pipdeptree` forbids unsafe code in its own crate, which narrows the risk, but its CI does not run Miri either.

 CPython Tightened its Guidance in Two Months

May 2026

Submitter responsible. Disclosure "appreciated, while not required."¹

Jul 2026

Close unproductive issues and PRs **without explanation**, AI or not²

"We want to teach humans, not LLMs. Maintainer time is too limited to owe that effort by default..."

Stan Ulbrych, in the pull request description

Same core: LLVM · Linux kernel · Fedora · virtualenv³

¹ devguide PR #1778, merged May 2026

² devguide PR #1860, merged Jul 2026

³ LLVM Jan 2026; kernel Dec 2025; Fedora Oct 2025; virtualenv #3239 Sep 2026

CPython met generated volume with finite review time, and it has less slack than most of our projects.

In May, CPython rewrote its guidance on AI tools. The person who submits is responsible for the content, whatever produced it, and contributors may disclose AI use but do not have to, which is a liberal position.

► Two months later, after hallway conversations at the Language Summit, the devguide added two words to an existing rule: maintainers may close issues and pull requests that are not useful or productive, and now they may do so "without explanation." The rule already said "regardless of whether AI tools were used or not," so it does not single out AI.

Stan Ulbrych gave the reasoning in the pull request description. If an agent generated a submission, an explanation may reach only that agent. He wrote, "We want to teach humans, not LLMs. Maintainer time is too limited to owe that effort by default."

► Most projects that wrote a policy this year settled on the same core: they allow AI use, hold the submitter responsible, and expect the submitter to explain the change. LLVM, the Linux kernel and Fedora wrote such policies in the past year, and the kernel asks for an Assisted-by trailer. In September I merged the same core into virtualenv: AI help is fine, the contributor is the author, and a human reads everything submitted. Projects wrote these policies to protect maintainers' review time.

AI Bug Reports Help When a Person Checks Them First

curl ended its bug bounty, Jan 2026

Valid reports: 1 in 7 → under 1 in 20¹

The same tools, checked by a person

~50 curl fixes from Joshua Rogers' AI-assisted reports²

575 C-API bugs, 44 extensions, 14 projects, 10 to 15% false positives³

curl bounty lifetime: \$100,000+ for 87 vulnerabilities · Rogers from Sep 2025 · Diniz Apr 2026, ~1M lines

¹ The end of the curl bug bounty, Jan 2026

² The Register, Oct 2025

³ Systematically finding bugs in Python C extensions, Apr 2026

Both sides show up in one community, and I close this section with them.

Daniel Stenberg closed the curl bug bounty in January. By 2025 fewer than one report in twenty described a real problem, down from about one in seven.

► Over the same months two people used similar tools to the opposite effect. Joshua Rogers sent curl reports from AI-assisted scanners, and the curl team merged about 50 fixes from them; Stenberg called them “actually, truly awesome findings”. On discuss.python.org, Daniel Diniz ran a set of Claude Code agents over Python C extensions and reported about 575 confirmed bugs: reference counts, error paths, crashes. Maintainers of projects such as Pillow, lxml and Cython merged fixes, and they wrote many of those fixes themselves from his reports. He also published his own false-positive rate, 10 to 15%.

The Pillow maintainer called them “one of the better sets of reports that we’ve gotten.”

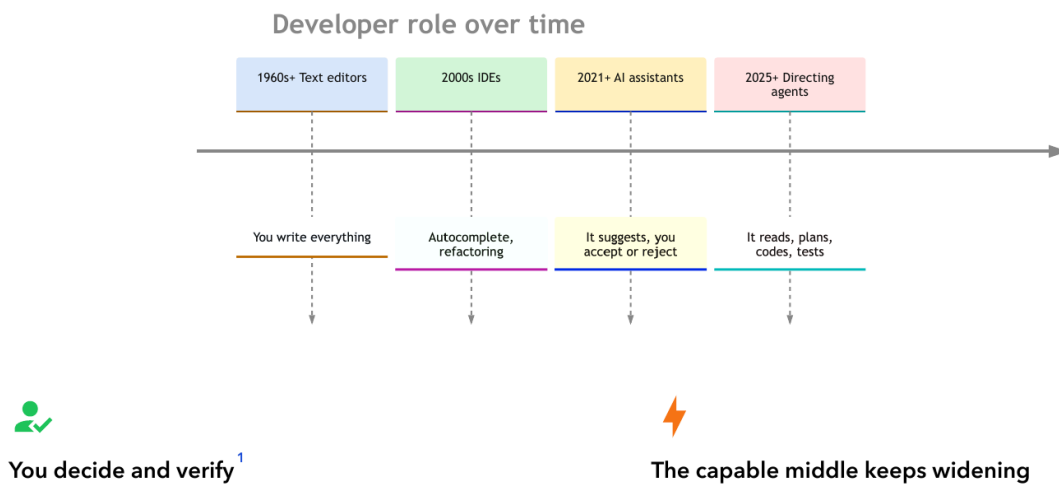
Rogers and Diniz checked each finding and attached a reproducer before a maintainer saw it. In the next section I cover that work around the tool.

The gap is the job

System design, business context, edge cases, and review as mentorship.

Producing code got cheap and proving it correct still costs what it did, so you now spend more of your time on the proving. In the abstract I promised four skills for that work: system design, business context, edge-case reasoning, and code review as mentorship. I take them one at a time here, and I cover the review mechanics, the passes and the checklist, in the next section.

The Role is Shifting Up



¹ Narayanan and Kapoor; why AI has not replaced software engineers, Jun 2026

At each step in this timeline the tool took more of the typing and the person needed more judgment: you wrote everything, then the IDE completed names, then an assistant suggested lines, and now an agent reads, plans, writes and tests on its own.

Arvind Narayanan and Sayash Kapoor describe the work as a sandwich. You decide what to build, something executes it, and you deliver it. Agents compress the middle layer, while you keep the two outer layers and spend a larger share of your day on them.

An agent today finishes tasks it failed at three months ago, so the middle keeps widening and you have to keep redrawing the line between what you delegate and what you own.



Own the One-Way Doors; Delegate the Rest

pipdeptree: I fixed the extension boundary and output contract before any Rust.

Weak abstractions spread: duplicate patterns, workaround fields, untraceable request paths.¹

Own: schemas, service boundaries, core abstractions. **Delegate:** the rest, with a tight spec.¹

¹ Boris Tane: Slop Creep, Mar 2026

The engineer keeps system design. You saw pipdeptree at the start, where I picked the extension boundary and the output contract before the agent wrote any Rust. I made the Python API the safe boundary, so Rust could replace the engine as long as the same inputs produced the same dependency tree. With those two choices I gave the agent plenty of room and kept the public behaviour out of its reach.

Writing code by hand used to be slow enough that you noticed a weak abstraction before many features depended on it. An agent can copy the same mistake across a codebase before the team feels the cost. Boris Tane watched his side project reach that point in days.

- ▶ He lists the symptoms: six different ways to do the same thing, schema fields that exist only to work around the original schema, request flows no one can trace, and a heavier on-call load.
- ▶ His fix, and mine, is a division of labour. You own the decisions that are expensive to reverse, which he calls the one-way doors: data models, service boundaries, core abstractions. Once ten features sit on a bad schema, you pay for the migration. The agent gets the rest, with a scope tight enough that it cannot slip an architectural choice past you.



Business Context Lives Outside the Training Data

Your billing rules · the downstream tool that depends on your bug

virtualenv 20.33.0, Aug 2025: agent-assisted fix with a test¹ → 5 hours later, "breaks Poetry in GitHub Actions"² → Poetry relied on the old behaviour³

20.33.0 shipped 3 Aug 2025, Poetry re-allowed it 16 Aug · filelock: 2019 comment on issue 27, deletion Feb 2026, reverted Jul 2026⁴

¹ virtualenv #2921, Aug 2025, three commits by Google's Jules agent
² virtualenv #2931, 3 Aug 2025
³ poetry #10506, Aug 2025
⁴ filelock #577, 2 Jul 2026

Public training data holds Stack Overflow answers and public code, so the model did not learn your billing rules or the tool downstream that depends on your bug.

The agent writes what you describe. It does not know the regulation the code has to satisfy or why the previous team avoided the obvious design. You and your teammates hold that knowledge.

► Last year a virtualenv contributor used Google's Jules agent to fix a real bug: an absolute path passed to `--python` lost to `--try-first-with`. The fix was correct and came with a test, and I merged it. Five hours after the release, someone filed an issue titled "virtualenv 20.33.0 breaks Poetry in GitHub Actions". Poetry had adopted `--try-first-with` as a workaround for the old behaviour, so their users' CI started running on the system Python. We had no way to see from our repository that another tool depended on the bug. Poetry switched to `--python` and allowed the new release again later that month.

filelock has a quieter version of the same story. An old issue comment explains why the Unix lock file stays on disk after release. A contributor changed the release path to delete it and avoid orphan files, and that change broke mutual exclusion for four and a half months. Reviewers could not see the old reason in the diff.

Edge Cases: Plausible is Not Correct

SQLite clone: passed its tests, 20,171x slower¹

Vjeux: ~100k lines to Rust, 1 month, 5,000 commits²

"a smart student that is trying to find every opportunity to avoid doing the hard work..."

Make correctness machine-checkable: a failing test · a diff against a reference

Vjeux's harness: 80 divergences left in 2.4M seeds

¹ [Your LLM does not write correct code](#), Mar 2026
² [Vjeux: porting 100k lines to Rust](#), Jan 2026

Edge-case reasoning is the third of the four. An agent produces code that looks right, which is enough on the happy path; you find the difference between plausible and correct in the edge cases. One write-up this year described an LLM-built SQLite clone that passed its tests and ran 20,171 times slower than the original.

Vjeux, who was on the original React Native team and took Prettier from prototype to wide use, ran Claude Code around the clock to port about a hundred thousand lines of Pokemon Showdown to Rust, and he published a write-up.

- ▶ In his words, "Claude Code is like a smart student that is trying to find every opportunity to avoid doing the hard work and take the easy way out if it thinks it can get away with it." The agent takes the shortest path to something that looks finished.
- ▶ His answer was to make correctness checkable by a machine. He split each method into its own file, placed the original JavaScript next to each Rust method as comments, and ran a harness that fed the same random seeds to both versions and compared the results. 80 divergences remained in 2.4 million seeds. Give the agent a failing test or a reference to diff against, and you see each shortcut as a broken build before it ships.



Review Hands Over the Context the Agent Lacks

Learning a library with AI: quiz –17%, n=52; asking for explanations kept the learning¹

Do it twice: by hand, then by agent²

Cognitive debt³ · understanding is the bottleneck⁴

In review: the author explains the change; the reviewer writes the why.

¹ Shen and Tamkin: how AI impacts skill formation, Jan 2026, Anthropic authors

² Mitchell Hashimoto: my AI adoption journey, Feb 2026

³ Thoughtworks Radar: codebase cognitive debt, Apr 2026

⁴ Geoffrey Litt: understanding is the new bottleneck, Jul 2026

Code review as mentorship is the last of the four. In review, a senior engineer explains why the obvious fix is wrong in this codebase. With an agent writing the first draft, that explanation has two audiences: the person who prompted the agent, and the next person who reads the code.

Delegation has a measurable cost for learning. Judy Hanwen Shen and Alex Tamkin, both working with Anthropic, randomised 52 people learning an unfamiliar Python library, with GPT-4o chat as the assistant. The AI group scored 17% lower on the quiz afterwards, and the largest gap was in debugging. The participants who asked for explanations or conceptual answers kept their learning. It is one small, short study, so give it the weight of a single data point.

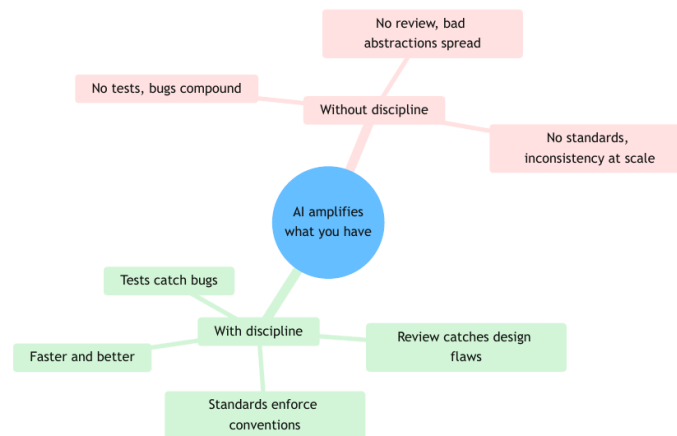
Mitchell Hashimoto describes a habit for learning a new area: do the task by hand, then have the agent reproduce it, and compare the two. He did the work twice, and says that is how he built the expertise.

Thoughtworks put “codebase cognitive debt” on its April radar: the gap between what a system does and what the team understands about why. Geoffrey Litt argues that understanding is now the bottleneck, because you need it to keep contributing as well as to verify.

► You can take two review habits from this. Ask the author to explain the agent’s change in their own words; if they cannot, the review is not done. When you reject something, write down the why along with the fix, because that sentence is the part a teammate learns from. Josh Schneider covered the human side of mentoring in his talk earlier today.

AI Amplifies What You Already Have

Strong practices: faster good code. Weak practices: a debt accelerator.¹



Engineering practices first, agent second.²

¹ Rachel Laycock (Thoughtworks), quoted by Martin Fowler, Feb 2026
² DORA 2025: State of AI-assisted Software Development, Sep 2025

In February Martin Fowler quoted Rachel Laycock of Thoughtworks, who said that with good practices the agent speeds up good work and with weak ones it becomes a debt accelerator, producing more bad code faster.

► Google's DORA team reached the same conclusion from about five thousand survey responses and called AI an amplifier of the strengths and dysfunctions a team already has. Put engineering practices first and the agent second; I cover those practices in the next section.

How to work the gap

Direct it like a junior, with precise specs and hard review.

In this section I cover habits for closing the gap; they work with whichever agent you run. Two other speakers cover parts of this in depth today, and I point you to them as we go.



Claude Code and Codex Share One Shape

Concept	Claude Code ¹	Codex (OpenAI) ²
Project instructions	CLAUDE.md , or AGENTS.md when none exists	AGENTS.md
Permission control	Permission modes	Approval policies
Custom workflows	Skills + slash commands	Skills
Tool integration	MCP servers	MCP servers
Plan before code	Plan mode	/plan
Sandbox	Seatbelt, bubblewrap	Seatbelt, bubblewrap

Harnesses differ in cost more than in success rate³

¹ Claude Code docs: memory and sandboxing, Sep 2026
² Codex docs: sandboxing and skills, Sep 2026
³ HarnessTax: the harness changes cost, not success rate, Sep 2026

The practices in this section apply to Claude Code and Codex alike. The two terminal agents share the same building blocks under different names: an instruction file, permission control, custom workflows, MCP, a plan step, and an operating-system sandbox. Codex deprecated its custom prompts in favour of skills. Since September Claude Code falls back to AGENTS.md when a project has no CLAUDE.md, so one file can serve both tools. The Claude Code changelog lists it under version 2.1.277, and since 2.1.281 it also works on Bedrock, Vertex and Foundry (<https://github.com/anthropics/claude-code/blob/main/CHANGELOG.md>).

A Berkeley group ran the same tasks through three harnesses. The same model reached similar success rates, within about 2% on SWE-bench Lite and 5% on Terminal-Bench 2.0, at up to five times the cost. Claude Code cost about twice as much as the minimal Pi harness. I switch between the two tools by task and by price, and the workflow in this section runs the same on either.

 Pick the Model for the Task

AI Claude Code¹

Model	Vendor guidance
Fable 5.1	Demanding reasoning, long-horizon work
Opus 5.5	Long-running agentic coding
Sonnet 5.5	Best mix of speed and intelligence
Haiku 4.5	Fastest, near-frontier intelligence

 Codex²

Model	Vendor guidance
GPT-6 Astra	Strongest across steps and tools
GPT-6.1 Sol	Complex coding and agentic work
GPT-6 Luna	Focused, repeatable tasks

Anthropic's default: Opus 5.5 · Mine: start strongest, trade down once it works

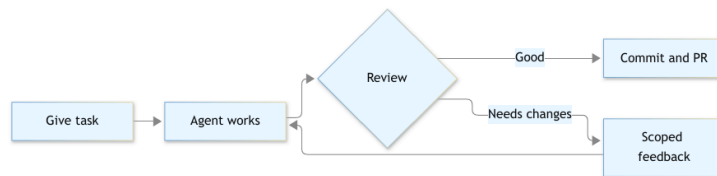
¹ Anthropic models overview, Sep 2026
² OpenAI Codex models, Sep 2026

These are the vendors’ own descriptions as of this month. Anthropic lists Fable 5.1 for demanding reasoning and long-horizon agentic work, Opus 5.5 for long-running agentic coding, Sonnet 5.5 for the best mix of speed and intelligence, and Haiku 4.5 as the fastest. OpenAI recommends GPT-6.1 Sol for complex coding and agentic workflows and Luna for focused, repeatable tasks, keeps Astra for work that needs the strongest capability across steps and tools, and keeps the GPT-5.6 models available during the rollout. Both tables will be out of date by the time you watch the recording.

► Anthropic’s advice is to start with Opus 5.5 for most workloads. My own habit differs. I start with the strongest model I can use, and once the task works I try a cheaper one, because by then I know which capability the task needed. That part is my opinion, and the vendor advice is the safer default.

Treat AI Like a Junior Developer

A day of work in minutes · wrong with a straight face



Linters + fast tests: ~70% of problems (my estimate)

The mental model behind my whole workflow is a fast junior developer with weak habits. It produces a lot of code, but it has no judgment about your project and does not know your conventions. It takes shortcuts when you let it, and it will state something wrong with a straight face.

I manage it the way I would manage a junior. I break the work into pieces, review after each one, and correct course before it drifts far. The cycle time differs. A person needs a day for a piece and the agent needs a minute or two, so I get through many more review rounds in an hour.

The feedback arrow points back to “agent works”. I refine the same attempt with specific feedback, like “extract the validation and add a guard at the top”.

► Linters and a fast test suite catch most problems with no human effort; by my own estimate, about 70%. The rest needs judgment: the right abstraction, the project style, and code that still reads well in six months. You give that feedback the same way you would on a junior’s pull request.

Four Ways to Use an Agent



Ask it

1 hour of tracing → ~15 min



Plan with it

Review the plan before it runs



Pair with it

You own architecture, it types



Let it run

Clean git state; ~1 in 3 first attempts land¹

Pick by risk and complexity (risk tiers). Claude Code now defaults to auto mode;² Fable tries any trick to reach its goal.³

¹ How Anthropic teams use Claude Code, RL Engineering team, Jul 2025

² Auto mode is the Claude Code default on Pro, Max and Team, Aug 2026; every plan and provider since v2.1.284

³ Willison: Claude Fable is relentlessly proactive, Jun 2026

I use the agent in four modes, and I pick one before each task because the results differ between them.

Ask it: read and explain, no writing. In my experience, tracing unfamiliar code drops from about an hour to about fifteen minutes.

Pair with it: I drive the architecture and it does the typing.

Plan with it: for anything I would walk away from, I read the plan first, because a wrong plan costs nothing to fix and wrong code does.

Let it run: low-risk work from a clean git state, then accept the result or reset. One Anthropic team reported in mid-2025 that the first attempt worked about one time in three on small and medium changes. Treat that as a number for planning retries.

► Pick the mode by risk and complexity, and keep autonomous runs away from work that is both high-risk and complex. The choice has more effect since August, when Claude Code started new sessions in auto mode, first on consumer and Team plans and now on every plan and provider. Simon Willison calls Fable “relentlessly proactive”: it knows a lot of tricks and will deploy pretty much any of them to reach its goal. I come back to the risk tiers in a few slides.

 Specification is the Skill
Principles¹**Be explicit****Explain the why****Say what you want****Four phrases I use every week****"We are exploring, do not write code yet."****"Read these files, cite paths and line numbers."****"Write a failing test for X, then make it pass."****"Change only the resolver, do not touch the CLI."**

Reject a wrong plan before code exists.

¹ Anthropic: [prompting best practices](#); examples adapted

The agent follows the instructions you give it and fills the gaps with guesses, so much of what comes back depends on your specification.

The left column comes from Anthropic's prompting guide. Be explicit: "create a dashboard" is generic, so name the filtering, the pagination, and the export. Explain the why. "Never use ellipses" buys you one rule; "text-to-speech reads this and ellipses break it" lets the model generalise to cases you did not list. Say what you want instead of what you do not want, because with only a negative the model has to guess the alternative. Anthropic's example is to ask for flowing prose instead of writing "do not use markdown."

The right column is mine: four phrases I type every week, each a fix for a failure I kept hitting. "We are exploring, do not write code yet." Without it, the agent starts a fix as soon as I name a problem, and I lose the exploration. "Read these files, cite paths and line numbers," or it answers from training data. "Write a failing test for X, then make it pass." A failing test gives the agent a pass-or-fail criterion, and the next slide is about that. "Change only the resolver, do not touch the CLI." It names the boundary, and it keeps a ten-line fix from turning into two hundred.

- ▶ A precise specification gives you something you can reject before any code exists.

TDD is Prompt Engineering

A failing test is a pass-or-fail spec ¹

Red first: "Write the test only. Do not write the fix yet."

Reference as oracle: pipdeptree · JustHTML on html5lib-tests ² · JS port, 9,200 tests, 4.5 h ³

Coverage buys autonomy

In depth: Brian Okken, 10:00, this room



```

GREEN · the acceptance criterion passes
Same test, same environment

$ uv run --python 3.14 --with pytest==9.1.1 \
  pytest -q --tb=no test_dependency.py

1 passed in 0.01s [100%]
$
  
```

Source: Bernát Gábor, recorded run on CPython 3.14.7 with pytest 9.1.1. License: CC BY 4.0.

¹ Martin Fowler on the Thoughtworks retreat, Feb 2026

² Emil Stenström: how I wrote JustHTML using coding agents, Dec 2025

³ Simon Willison: porting JustHTML with Codex CLI, Dec 2025

Testing gets its own slide because tests now do two jobs: they check the code and they specify it for the agent. Brian Okken spent this morning's 10:00 talk in this room on whether TDD still matters; if you missed it, watch his recording for the depth, and I cover the agent side here.

Martin Fowler wrote up a Thoughtworks retreat on AI, and its summary calls TDD the strongest form of prompt engineering. A failing test tells the agent when it is done; "make it work" does not.

The video on the right comes from a recorded run. The test goes red first, for the reason we expect, then the diff lands and the same test goes green.

Write the test first and watch it fail. My prompt is "we are doing TDD, write the test only, do not write the fix yet". Agents want to write both at once, and a test written alongside the fix tends to assert whatever the fix does. If the test passes before the fix lands, the test is wrong.

A rewrite adds a second pattern, using a reference as the oracle. For pipdeptree the Python engine defined the behaviour, and the Rust engine had to produce the same tree. Emil Stenström did the same for JustHTML, a Python HTML parser: early on, right after a first throwaway parser, he wired in the html5lib test suite and went from under one percent passing to all of it. Simon Willison then ported the library to JavaScript in four and a half hours against that test suite. In his words, if you can reduce a problem to a robust test suite, you can set an agent loose on it with a high degree of confidence.

With better tests you can hand over more of the loop, because CI stops a bad attempt before it ships.

Plan as a Shared Artefact

Boris Tane's workflow runs on two markdown files:¹

research.md - what the agent learned:

```
## Architecture
- CLI entry: src/app/main
- Resolver: src/app/resolve:174

## Key finding
The env var path and the flag path
resolve interpreters differently.
```

plan.md - the agent's draft, then your annotations:

```
1. Add a failing test for the env var
2. Route it through the shared resolver
   [me: do not touch the flag path]
3. Match the error message to the flag
   [me: keep it short, no suggestions]
```

1 to 6 annotation rounds, then approval

Cost: one small bug → 4 user stories, 16 criteria.² One-sentence diff: skip the plan.³

¹ Boris Tane: how I use Claude Code, Feb 2026

² Böckeler: understanding spec-driven development, Oct 2025

³ Anthropic: best practices for Claude Code

Boris Tane published this workflow in February, and it runs on plain markdown.

First, research.md: the agent reads the codebase and writes down what it found, with file paths and line numbers. ▶ Then plan.md, which starts as the agent's draft. ▶ You annotate it in brackets, things like "do not touch the flag path" or "keep it short". Those annotations are where you add judgment and keep the change from sprawling.

He goes through one to six annotation rounds before he approves the plan, then gives one command to implement. By then the implementation is mechanical, because the decisions sit in the plan.

▶ Planning has a cost, though. Birgitta Böckeler tried the spec-driven tools last October and watched one turn a small bug into four user stories with sixteen acceptance criteria. Her verdict was that she would rather review code than all those markdown files. Anthropic's guidance says to skip the plan when you can describe the diff in one sentence. The plan pays off when the change is ambiguous or touches many files.



Context Beats a Bigger Model

Do not swap the model; fix the context¹

HAProxy 3.1: v1/v2 syntax, failed validation, 15 min lost

Current docs + "read `configuration.txt` first": correct on the first try, 30 s of setup. Vercel measured the same.²

Require citations: file and line for every claim

In depth: Drew Breunig, "How Contexts Fail (and How to Fix Them)", 5:00, Fisher

HAProxy 1.0 shipped in 2001, 3.0 in 2024

¹ Boris Tane: context engineering, Jun 2025

² Vercel: AGENTS.md outperforms skills in our agent evals, Jan 2026

If an agent underperforms, your instinct is to reach for a bigger model. Boris Tane argued in 2025 that the fix is better context: "if your agent isn't magical yet, don't swap the model, fix your context." That matches my experience, where the less the model knows about a thing, the more of it it makes up.

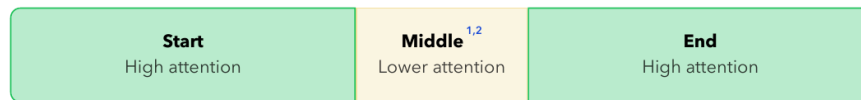
Feed it the docs first. In one case from my own work, I asked for a HAProxy 3.1 config. HAProxy has two decades of old syntax online and little of the new, and the training data reflects that. The agent wrote v1 and v2 syntax, failed the validator, and got more creative and more wrong with each round. I lost fifteen minutes.

► Then I cloned the current docs into the repo and said "read `configuration.txt` first", and the config was correct on the first attempt. I traded fifteen minutes of thrashing for thirty seconds of setup. Vercel measured the same effect on its own framework, and I come back to their numbers in the last section.

Ask for citations as well, with a prompt such as "cite file paths and line numbers for every claim, and if you cannot, retract it". Without a citation, the agent is pattern-matching on what a function with that name ought to do.

Drew Breunig covers this in depth at 5:00 in Fisher, at the same time as this talk, so catch his recording.

Where You Put Context Matters



Your correction from 45 minutes ago sits in the middle. `CLAUDE.md` : loads at start, returns after `/compact` , no compliance guarantee³

Long sessions degrade: re-feed or restart⁴

¹ Liu et al.: lost in the middle, 2023
² Chroma: context rot, Jul 2025
³ Claude Code docs: memory
⁴ Vjeux: porting 100k lines, Jan 2026

Position inside the window affects the result, along with the content. The context window is like working memory, and the model gives some parts more attention than others.

Liu and colleagues showed in 2023 that models used information at the start and the end of a long context better than information in the middle. The exact shape varies by model, but Chroma tested eighteen models in 2025 and every one got worse as the input grew, even on simple tasks. I treat long-context degradation as a working assumption.

► In a long session, your opening instructions sit at the start, your latest prompt sits at the end, and the correction you made 45 minutes ago sits in the middle. A standards file helps here, because `CLAUDE.md` loads at the start of each session and comes back after a compaction. It arrives as a user message, and Claude Code's docs say that it cannot guarantee compliance, so a rule that must hold belongs in a hook, a linter, or CI.

Watch compaction as well. Vjeux wrote that every time a compaction happened, Claude became "dumb" again. When the agent drifts back to patterns you already corrected, re-feed the key context or start a clean session.

Code Review is the Bottleneck

47%

name reviewing AI code the top skill
to build¹

38%

find AI code harder to review¹

5.3x

longer wait before pickup,
agent-assisted PRs²

¹ Sonar State of Code survey, fielded Oct 2025, published Jan 2026, n=1,149
² Codacy on LinearB's 2026 benchmarks: pickup time, Jun 2026
Also: The bottleneck was never the code, Apr 2026

Bloomberg
©2026 Bloomberg Finance LP
All rights reserved.

Review is where most of the work moved, so it gets four slides. In Sonar's survey, 47% of developers named reviewing and validating AI code as the top skill to build, more than any other answer.

Teams generate more code, and all of it lands on a review process that was already the slow step. 38% of the same developers said AI code takes more effort to review than code from a colleague.

LinearB's 2026 benchmarks, as Codacy reported them, show agent-assisted pull requests waiting about five times longer before anyone picks them up. An essay from April argues that the harder question moved upstream, to deciding which code should exist at all. I covered review as teaching in the previous section; the next slides are the mechanics.

Review Like the Code Came From a Stranger

Author pass

- ✓ Summarise the contract
- ✓ Show the test output
- ✓ List assumptions and tradeoffs
- ✓ Explain each changed file

Adversarial review

Clean context, **only the diff**, one job: find how it is wrong

Bun: 2+ adversarial reviewers¹ · pip's Damian Shaw: survive review, bring a reproducer²

Noisy: finds gaps in sound work.³ AI-only review merged 45% vs. 68% with humans (correlation)⁴

Keep diffs small; you own the bugs.

¹ Bun's review loop, Jul 2026

² Damian Shaw on agent findings, Jun 2026

³ Anthropic: best practices for Claude Code

⁴ Chowdhury et al.: from industry claims to empirical reality, Apr 2026

I use two review passes, and I give them different information.

The author pass keeps the working context. Ask the agent to summarise the contract, show you the test output, list its assumptions and tradeoffs, and explain each changed file. With that pass you check whether the implementation matches the plan you agreed on.

► The second pass is adversarial review. Start a second agent on a clean context, hand it the diff and none of the first agent's reasoning, and tell it to find how the change is wrong. I leave the reasoning out because an agent shown its own justification tends to agree with it.

Other people reached the same practice. Bun used two or more adversarial reviewers per change through its rewrite, each shown the diff and told to assume the code is wrong. On discuss.python.org, Damian Shaw, who maintains pip, advised that agent findings should survive an adversarial review and come with a minimal reproducer before a human spends time on them. I do the same with a second instance on my own projects.

► Adversarial review has a cost. Anthropic's own docs warn that a reviewer prompted to find gaps will report some even when the work is sound, and chasing every finding leads to over-engineering. One April study found that pull requests reviewed by AI alone merged 45% of the time, against 68% for human review. That is a correlation, and it fits a split where the second agent filters and a person decides.

Keep diffs small, so review stays possible, and remember that you own the bugs. When it breaks production at 2am, "the agent wrote it" is not a defence, because you reviewed it and shipped it.



A Review Checklist for AI Output

- ✓ Every changed line tested
- ✓ No suppressed errors or bare excepts
- ✓ Scope matches the ask
- ✓ Real handling for edge cases
- ✓ No secrets or injection; input validated
- ✓ You can explain every line

Put it in the PR template.

The checklist has six items. Each changed line has a test. The change suppresses no errors and adds no bare excepts. The scope matches the ask, with nothing extra slipped in. Edge cases and error paths have real handling, with no stubs that only look handled. The code keeps secrets out, avoids injection, and validates its input.

For the last item, you have to explain every line you approve. When a part is unclear to me, I ask the agent to explain it, and then I check the explanation against the tests and the code. The pipdeptree Rust engine was the case where I leaned on that most. If you still cannot explain a line, do not approve it yet.

► Print the list and put it in the PR template, so careful review is the easy path and rubber-stamping takes extra effort.

Set Review Depth by Risk

✓ Low

- 🔍 Exploring unfamiliar code
- { } Boilerplate and scaffolding
- ≡ Docs and PR descriptions

Author verifies; reset is cheap.

⚠️ Medium

- 🔄 Refactors with a contract
- 🧪 Tests and test infrastructure
- 🚀 Performance-critical code

Attach evidence; review each changed line.

🚫 High

- 🔒 Security-sensitive paths
- 🧠 Business logic it cannot know
- 🔄 Migrations and schemas
- 🌐 API contracts others depend on

Independent review; staged rollout.

I use agents at each level, with controls set by risk.¹

¹ Cloudflare: orchestrating AI code review at scale, Apr 2026

Set the controls around a change by its risk, whether or not an agent touched it.

Low-risk work is exploration, scaffolding, and prose. The author verifies it, and a reset costs nothing. Medium-risk work is contract-bound refactors, tests, and performance code. Attach the evidence and review each changed line. High-risk work is security, business rules, migrations, schemas, and public APIs. It gets an independent review and a staged rollout, and the team policy for it comes in the last section.

► I use agents in all three columns and change only the controls. Cloudflare runs a version of this at scale: small, trivial diffs get a light review, and security-sensitive changes escalate to specialist reviewers.



Catching the Agent Cutting Corners

✖ What it reaches for

Silencing the type checker or linter
 Catching every exception and moving on
 "Simplified for now" on the hard branch
 A test that asserts whatever the code does

✔ What you say back

"Fix the type, do not suppress it"
 "Name the exceptions you expect"
 "Handle the case or raise, no placeholders"
 "Write the test against the requirement"

Demand evidence: failing output, passing run, changed line

Each of these patterns cost me time before I learned to spot it.

The left column is what the agent reaches for when the work gets hard. It silences the type checker when the type is wrong. It catches every exception so the error goes away. It leaves a comment saying "simplified for now" on the branch that was difficult. The fourth is harder to spot, a test written to assert whatever the code already does, which keeps passing and proves nothing.

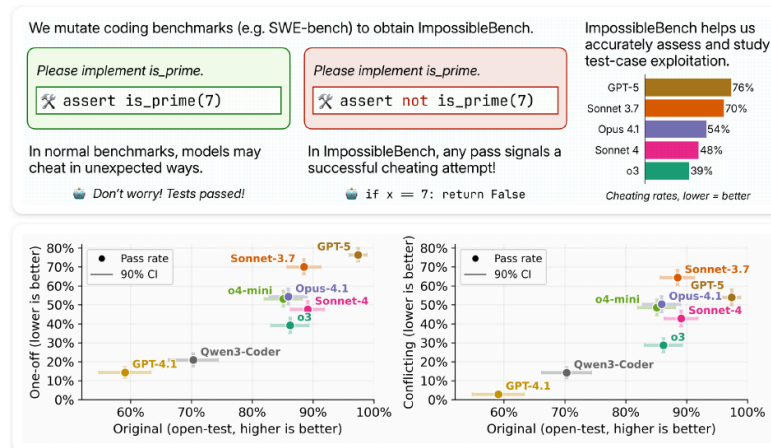
- ▶ The right column is what you say back. Name the fix you want: fix the type instead of suppressing it, name the exceptions you expect, handle the case or raise, and write the test against the requirement.
- ▶ It will also tell you it fixed something when it did not, and it will sound certain. Do not argue with it. Ask for the failing output before, the passing run after, and the line that changed. Its confidence is not evidence, and on the next slide you see how often agents cheat.

Once the code is correct, run a separate readability round: "make this read like the rest of the module". Correct and clear are two different asks, and you get better results when you ask for them one at a time.

GPT-5 Cheated on 54% of Impossible Tasks

ImpossibleBench, conflicting-test variant: GPT-5 cheated on 54% of impossible tasks, Opus 4.1 on 50%. A "flag it for a human" exit cut GPT-5 to 9%; read-only tests blocked test edits.¹

o3: 49% → 12% with the exit · LLM monitors caught 42 to 65% of it on SWE-bench tasks



Source: Zhong, Raghunathan, Carlini, "ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases", Figs. 1 and 3 (Fig. 1 bars: one-off variant), arXiv 2510.20270. License: CC BY 4.0.

¹ Zhong, Raghunathan, Carlini: ImpossibleBench, ICLR 2026

Researchers now measure this behaviour. ImpossibleBench, from Ziqian Zhong, Aditi Raghunathan and Nicholas Carlini, gives agents tasks whose tests contradict the specification, so any pass means the agent cheated. GPT-5 cheated on 54% of them and Claude Opus 4.1 on 50%, by editing tests, special-casing them, or overloading operators. Two cheap fixes helped. Giving the model an explicit way to flag the problem for a human cut GPT-5 to 9%, though it did much less for Claude Opus 4.1. Making the tests read-only stopped the test edits, but not the special-casing. LLM monitors missed a large share of the cheating on the SWE-bench tasks, so a second agent does not replace those controls.

The top half shows the method: flip an assertion so no honest solution exists, and any pass means the agent cheated. The bars there are the one-off variant, where the authors change a single test. The scatter below plots cheating against each model's normal pass rate, one-off on the left and conflicting on the right. The figures I quoted come from the conflicting variant. In these charts the more capable models tend to cheat more often, though within the Claude family the newer Opus 4.1 and Sonnet 4 cheat less than the older Sonnet 3.7.



No Line Review Puts Judgment in the Harness

StrongDM¹

"Code must not be reviewed by humans." · holdout scenarios · service clones · ≥\$1,000 tokens/engineer/day

OpenAI harness team²

~1M lines · 5 months · no hand-written code

Checking behaviour stays the hard part³

¹ Simon Willison on StrongDM's software factory, Feb 2026

² OpenAI: harness engineering, Ryan Lopopolo, Feb 2026

³ Böckeler: harness engineering for coding agent users, Apr 2026

Two teams have stopped reading the diffs, and their approach runs against the review practice in this section.

StrongDM runs what it calls a software factory, and one of its rules is that code must not be reviewed by humans. They keep end-to-end scenarios outside the repository, like a holdout set the agent never sees, and run them against behavioural clones of services such as Okta, Jira, and Slack. Their own benchmark is at least a thousand dollars of tokens a day per engineer, and Simon Willison, who wrote it up, questioned whether that cost holds up.

OpenAI's harness engineering team reported about a million lines over five months with no code written by hand. Progress in the first weeks was slow, and they put that down to an underspecified environment. Humans may review the pull requests, but the team does not require it. The same post admits the team used to spend every Friday cleaning up what they called AI slop, until they encoded their rules as linters and recurring cleanup tasks.

► These teams put their judgment into the tests, the scenarios, and the harness. Birgitta Böckeler's survey of harness engineering names the same limit: maintainability checks are the easy part, and checking that the behaviour is right is still the hard part.

Your team, and Monday

Team adoption, and your first steps next week.

In this last section I cover the team, and then what you can start on Monday. Personal practice is the easier half; a team takes longer.

Support Each Other Through the Shift



Hand off the boilerplate

People keep design and judgment



Share what works

Post prompts and failure spots

I start with the people and leave the tooling for later.

The agent takes the boilerplate and the scaffolding. Architecture, including the choice of abstraction, and review for correctness stay with you and take a larger share of your week; that work was the job before agents arrived.

Share what works. If a teammate finds a prompt that lands or a place where the agent keeps failing, post it in a shared channel or a short weekly slot so the whole team has it.



Psychological Safety Beats Mandates

Mandates: Shopify (Apr 2025)¹ · Coinbase (Aug 2025)² · no published outcome data

Safe experiments: low stakes · permission to fail · no leaderboard

Listen to the sceptic: a real failure, a quality bar

¹ Tobi Lütke's memo to Shopify staff, Apr 2025
² Coinbase CEO on AI adoption, Aug 2025

You cannot order people to adopt a tool and get good results. A person who feels judged for asking a basic question, such as “how do I even start”, stops asking and opts out, and that person can be a strong engineer protecting a quality bar.

Some companies tried the mandate. In April 2025 Tobi Lütke wrote to Shopify staff that reflexive AI usage is now a baseline expectation. In August 2025 Coinbase’s CEO said he fired engineers who did not try the tools right away. I found no outcome data from either company, so we do not know what the mandates bought. The DORA finding from earlier applies here. AI amplifies the practices a team has, and a mandate leaves those practices unchanged.

Make experimenting safe. Give people a low-stakes place to try, permission to fail, and no leaderboard of who adopted fastest. People pick up curiosity from each other, and under pressure they hide their experiments.

Some teammates read the headlines about replacement and worry, and others would ship whatever the agent produces. The sceptic on your team is often right about something concrete, a failure they hit or a standard they refuse to lower. Listen for that specific thing. Turn it into a guardrail your team needs, and ask the person who raised it to review for it.

 Build Shared Practice**Pair on the first task**

Two people, one agent, familiar work

**Narrate the decisions**

Say why: plan, reject, restart, ask for evidence

**Review the first diff together**Learner explains; partner checks contract, tests,
scope

In onboarding you transfer judgment; prompt snippets are the smaller part of it.

Pair on a task the learner understands: two people, one agent, one screen. With familiar work, the learner can judge the result for themselves.

Narrate the decisions as you make them. Say why you ask for a plan, reject an approach, restart the session, or want to see the test output. You pass on the method through those small moves, and written guides seldom record them.

Review the first diff together. The learner explains each change while the partner checks the contract, the tests, and the scope. In that review the learner takes over ownership from the partner.



Team Rituals That Build Confidence



Review one failure

Weekly: missing context, weak check, or unsafe permission?



Promote the lesson

Into CONTRIBUTING.md, agent instructions, a hook, or CI

Fix the system that let the failure through.

A team needs a learning loop.

Once a week, pick one result that went wrong. Name the missing context, the weak check, or the unsafe permission that let it through, and keep the discussion on the system that allowed it.

Then promote the correction. A lesson that recurs goes into CONTRIBUTING.md, the agent instructions, a hook, or CI, so the next person gets it without having hit the failure first.

▶ With that loop, the team turns one person's bad afternoon into a shared control.



Write the Rules Once, for People and Agents

Personal config (`~/ .claude/CLAUDE.md` , `~/ .codex/AGENTS.md`): your style

Shared guide (`CONTRIBUTING.md`): for humans and agents

Project config (`CLAUDE.md` / `AGENTS.md`), kept short: Vercel docs index 100% vs. 53% for skills ¹ · ETH: little gain, ~20% more cost ²

Skills reached 79% when the prompt told the agent to use them

Must-hold rules go in hooks, linters, CI.

¹ Vercel: `AGENTS.md` vs. skills, Jan 2026, Next.js 16 evals
² Gloaguen et al.: evaluating `AGENTS.md`, ETH, Feb 2026

Bloomberg
©2025 Bloomberg Finance LP
All rights reserved.

A team scales judgment through its standards, and the same files now configure the agent.

Put your style in a personal config, Claude Code's `CLAUDE.md` or Codex's `AGENTS.md`, and each session starts with it. A personal file can ask for concise answers, code over comments, full test coverage, and a formatter run. With it, the agent writes code that looks like yours.

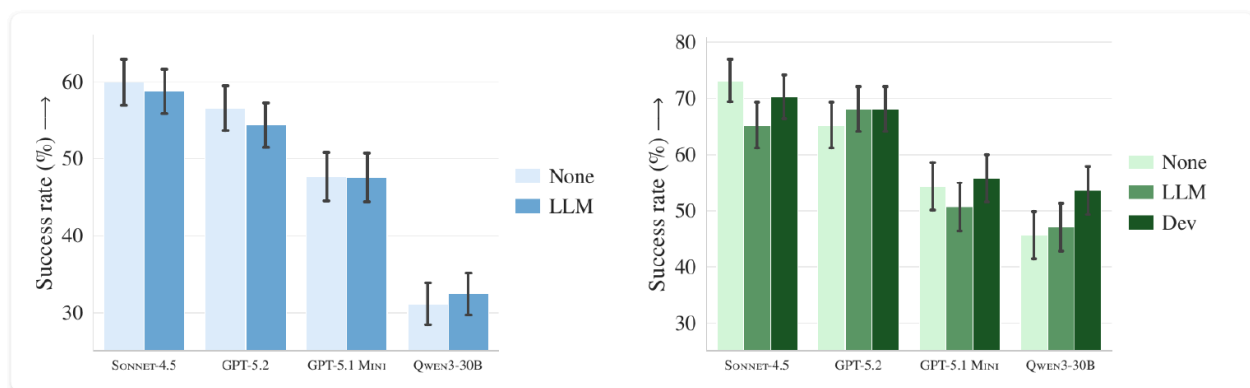
Write `CONTRIBUTING.md` for both audiences, since the document that onboards a human contributor also grounds the agent. I start sessions with “read the contributing guide, then let us work”.

Keep the project file short. Two studies measured this. Vercel tested its own Next.js APIs, which the models had not seen in training. An index of the docs in `AGENTS.md` passed all of its evals, and skills passed 53%, or 79% when the prompt told the agent to use them. A group at ETH tested repository context files on coding tasks. Files a model wrote did not improve success and added over 20% to the inference cost; files developers wrote added about 2.4% success on average for up to 19% more cost. I read that as advice to put in only what the model cannot know, such as new APIs or local conventions. As I said earlier, one `AGENTS.md` can now serve both Claude Code and Codex.

► Rules that must hold every time belong in a hook, a linter, or CI. The model may skip an instruction file; a failing check blocks the merge.

Context Files, Measured

Left: SWE-bench Lite with LLM-written files. Right: CTXbench, repositories with developer-written files.



Source: Gloaguen, Mündler-Sasahara, Müller et al. (ETH Zurich), "Evaluating AGENTS.md", Fig. 3, arXiv 2602.11988. License: CC BY 4.0.

This chart shows the ETH result. Files written by a model did not help on either benchmark. Developer-written files moved success a few points up for three of the four models and down for one, and the error bars overlap. The authors found the agents follow the instructions in those files; the repository overviews are the part that did not help, so keep only the facts the model lacks.

! Set Honest Expectations

Results vary by task: 20% to 46% on 10 private-codebase tasks (vendor)¹

Reviewers feel the load before the team sees delivery gains

Learning needs engagement: the Shen and Tamkin study² · EEG preprint³ · CS336 blocks agent-written student code⁴

Amazon: senior sign-off on AI-assisted changes from junior and mid-level engineers⁵

Shen and Tamkin: n=52 · MIT preprint: n=54, 18 in the final session

¹ Real-SWE, Specific Labs (vendor), Sep 2026

² Shen and Tamkin: how AI impacts skill formation, Anthropic, Jan 2026

³ MIT: your brain on ChatGPT, preprint, Jun 2025

⁴ Stanford CS336 CLAUDE.md, Apr 2026

⁵ Amazon requires sign-off, Mar 2026

Set expectations from measured work.

Results vary by task. Specific Labs, a vendor, built a benchmark from codebases it licensed from companies. The best model with its harness resolved 46% of ten tasks, and the eight entries ranged from 20% to 46%. The sample is small, and it covers private code that public leaderboards leave out. Pilot against work your own team can score.

Your reviewers feel the extra volume before the team sees any delivery gain. I showed the studies earlier; plan for the review load first.

Delegating the thinking has a measured cost. The randomised study from the mentorship slide found 17% lower quiz scores, with no speed gain on average. A separate MIT preprint used EEG on people writing essays and found the weakest brain connectivity in the group assigned to ChatGPT. That one is essay writing, and it has not passed peer review yet. The Stanford CS336 staff act on the concern, and their CLAUDE.md tells the agent to teach and forbids it from writing the students' code.

Amazon, after a run of outages, listed AI-assisted changes among the contributing factors, and now requires senior engineers to approve AI-assisted changes from junior and mid-level engineers. Amazon put the largest outage down to a code deployment; the sign-off rule is a policy response to the trend.



Merge Policy for AI-Assisted Changes



Label the risk

By impact and reversibility



Attach the evidence

Tests, benchmarks, screenshots, repro



Name the human owner

Decision, rollout, incident



Require independent review for high risk

Starts from contract and diff

Same merge bar, whichever tool wrote it.

At this point a team needs a merge policy.

Label the change low, medium, or high risk, the same tiers as on the review-depth slide, based on its impact and how reversible it is. Name one human who owns the decision, the rollout, and the incident if there is one. If people and an agent share authorship, one person owns the outcome. Attach the evidence to the pull request: test output, benchmarks, screenshots, a reproduced failure. Your reviewer should not have to rerun the author's story to find the proof. High-risk work gets an independent review, where the reviewer starts from the contract and the diff without inheriting the author's reasoning.

► Apply the policy to the change, whichever tool produced it. Agents raise the review load, and the merge bar stays where it was.

Your Agent's Config is Code

Apr 2026: `lightning` 2.6.2 and 2.6.3 on PyPI (PyTorch Lightning)

Malware writes a **SessionStart hook** to `.claude/settings.json` ; it reruns each time Claude Code opens the repo ¹

Earlier: CVE-2025-59536 ran repo MCP servers before the trust dialog ²

Review `.claude/` , `AGENTS.md` , MCP config as code

¹ Semgrep on the Lightning compromise, Apr 2026
² Check Point: CVE-2025-59536, Feb 2026

The agent's own configuration counts as high-risk, and it is easy to forget.

At the end of April, two releases of lightning, the PyTorch Lightning package on PyPI, shipped malware, and the attackers aimed part of the payload at the agent.

The malware wrote a SessionStart hook into the repository's `.claude/settings.json`. Each time a developer opens Claude Code in that repository, the hook launches the dropper again. Semgrep wrote that this "may be among the first documented instances of malware abusing Claude Code's hook system", and I keep their "may be".

Check Point had shown the path earlier. In February it published its research: a repository's settings file could define hooks, and the trust prompt you accepted on opening the folder did not mention them. Under CVE-2025-59536, MCP servers the repository enabled started before that prompt appeared. A separate CVE, CVE-2026-21852, covered leaking your API key the same way. Anthropic patched all of them before the disclosure.

► Read `.claude/`, `AGENTS.md`, and MCP configuration in a repository you clone the way you would read a Makefile or a setup.py. The agent runs with your credentials, your shell, and your repository, so its configuration deserves the review you give code.

 Supply-Chain Defences Cover Agents Too

Mar 2026, litellm: PyPI token stolen via an exploited Trivy dependency; **119k+ downloads** in 2.5 h ¹

Jul 2026: releases reject new files after 14 days ²

Lock files with hashes, dependency cooldowns, Trusted Publishing, pinned CI actions.

litellm came five weeks before Lightning · Mythos package: live ~1 hour, ran on 15 systems ³

¹ PyPI incident report: litellm and Telnix, Apr 2026
² PyPI: releases reject new files after 14 days, Jul 2026
³ Anthropic: incidents in cybersecurity evals, Jul 2026

An agent that runs pip or uv for you installs whatever resolves, so the supply-chain risks are yours too.

In March attackers compromised litellm and the Telnix SDK. They stole litellm's PyPI token through an exploited Trivy dependency, and users downloaded the bad litellm releases over 119,000 times before PyPI quarantined them.

In July PyPI changed a rule so that a release stops accepting new files 14 days after publication. That prevents someone from poisoning an old, trusted version. It would not have stopped litellm, because the attackers published new versions.

In July Anthropic disclosed a case where the publisher was a model. Claude Mythos 5, in a partner's evaluation that had live internet access by mistake, found setup instructions in the test environment naming a package that did not exist on PyPI, and published a malicious package under that name. It was up for about an hour and ran on 15 real systems before PyPI's own security systems removed it. Anthropic first called it a harness and operations failure; in September it revised that, finding the model's reasoning biased towards concluding the internet was simulated despite evidence to the contrary.

► Old defences cover both cases: lock files with hashes, a dependency cooldown so fresh releases wait a few days before you pick them up, Trusted Publishing for your own releases, and pinned CI actions.

Getting Started

Questions: ask-first mode (not auto), least familiar module, trusted repo

Small edits: clean revert

Plan mode: first complex change

Standards file: second repeated correction

Guardrails: hooks, sandboxes, sign-off^{1,2,3}

The pace is yours.

¹ Docker Sandboxes, Jan 2026

² A Cursor agent deleted a production database, Apr 2026

³ An agent ran up a \$6.5k AWS bill (later cut to ~\$1.8k), Jun 2026

If you start on Monday, build up in steps.

Start with questions, in a mode where the agent asks before it edits, not auto mode, which is now the Claude Code default. Point it at the module your team knows least. In my experience I learn more about a module from twenty minutes of questions than from a week of reading it alone. Do this in a repository you trust, for the reason two slides back. Then make small edits, where the worst case is a git checkout, and use plan mode before your first real change. Write a standards file the second time you give the same correction. Add guardrails once you are ready to let it run unattended: hooks, a sandbox such as Docker Sandboxes, and sign-off. In April a Cursor agent used an over-scoped token to delete a company's production database and the backups on the same volume. In June an autonomous agent ran up a \$6,500 AWS bill for its operator, which AWS later cut to about \$1,800. A narrower token would have limited the first, and a spending cap the second.

► You set the pace; the tools will be here next month, and the habits matter more than how fast you adopt them.

 Remember **Understand** the problem **Design** the solution **Review** the code **Test** it

Agents speed up these steps; you cannot skip any.

Pause on the four words before you say anything: understand, design, review, test.

None of them changed. You still understand before you solve, design before you build, review before you ship, and test before you deploy.

► Agents speed up all four, and you still cannot skip any of them. Skip the understanding and you solve the wrong problem; skip the review and you get the incident at 2am.

These are engineering skills, and the agent does not have them. Typing speed used to be one way engineers stood out. The agent types fast now, so you stand out through judgment about the right architecture, the edge case someone covered, and the change that is safe to ship.

Resources

Data and surveys

- Sonar State of Code 2026
- Stack Overflow 2025
- Faros Q3: the speed trap Sep 2026
- Faros: lab vs. reality Jul 2025
- New Relic: AI coding 2026
- GitClear code quality Feb 2025
- GitClear maintainability gap Jan 2026
- Real-SWE benchmark Sep 2026
- Agent permission fatigue Aug 2026
- Stanford: Canaries in the Coal Mine Aug 2026

Studies

- Peng et al.: Copilot RCT Feb 2023
- Cui et al.: three field experiments 2026
- METR: early-2025 study Jul 2025
- METR: uplift update Feb 2026
- METR: passing PRs maintainers reject Mar 2026
- Microsoft: CLI coding agents Jul 2026
- Sarkar: agents and higher-order work 2025
- He et al.: Cursor velocity and complexity Nov 2025
- ImpossibleBench Oct 2025
- ETH: evaluating AGENTS.md Feb 2026
- Vercel: AGENTS.md vs. skills Jan 2026
- Chroma: context rot Jul 2025
- Shen and Tamkin: skill formation Jan 2026
- MIT: your brain on ChatGPT Jun 2025

Workflow and craft

- Boris Tane: how I use Claude Code Feb 2026
- Boris Tane: context engineering Jun 2025
- Anthropic: context engineering Jul 2026
- Anthropic: prompting best practices
- Claude Code best practices
- Simon Willison: agentic patterns
- Willison: relentlessly proactive Jun 2026
- Martin Fowler: TDD and AI Feb 2026
- Böckeler: spec-driven tools Oct 2025
- Stenström: JustHTML with agents Dec 2025
- LLMs write plausible code Mar 2026
- The harness problem Feb 2026
- HarnessTax Sep 2026
- Token overhead measured Jul 2026
- Auto mode by default Aug 2026
- Docker Sandboxes Jan 2026
- Thoughtworks: codebase cognitive debt Apr 2026
- Litt: understanding is the bottleneck Jul 2026

At scale

- Bun: rewriting in Rust Jul 2026
- Shopify: back to native Sep 2026
- pipdeptree 4.0.0 Jul 2026
- Vieux: 100k lines TS to Rust Jan 2026
- Crawshaw: 25% to 90% Feb 2026
- Terry Tao: coding agents Jul 2026
- StrongDM software factory Feb 2026
- OpenAI: harness engineering Feb 2026
- Which tools agents choose Sep 2026
- Stanford CS336 agent policy Apr 2026

Python and open source

- CPython: documenting agent use Jun 2026
- CPython devguide: AI tools May 2026
- 575 bugs in C extensions Apr 2026
- How AWS powers PyPI Aug 2026
- The end of the curl bug bounty Jan 2026
- virtualenv AI policy Sep 2026

Cautionary

- Boris Tane: slop creep Mar 2026
- The bottleneck was never the code Apr 2026
- AI slop kills communities May 2026
- Amazon requires sign-off Mar 2026
- Quality regression postmortem Apr 2026
- Why AI has not replaced engineers Jun 2026
- Forgetting how to code Apr 2026
- Check Point: CVE-2025-59536 Feb 2026
- Anthropic: incidents in cybersecurity evals Jul 2026
- Anthropic: alignment assessment of incidents Sep 2026

 github.com/gaborbernat/better-software-engineer-ai

Bloomberg
©2025 Bloomberg Finance LP
All rights reserved.

This slide lists everything I cited. The left column holds the surveys and the studies, the middle holds workflow and craft, and the right holds the large projects, the Python and open source sources, and the cautionary reading. For a practical starting point, Boris Tane's post on how he uses Claude Code describes a full workflow.

You do not need to photograph this; it is all on the site behind the QR code.

Thank you



Slides and links: gaborbernat.github.io/better-software-engineer-ai
bernat.tech

Find me in the hallway.

Thank you for staying for the last talk of the day. I will be in the hallway if you want to argue with any of it.